

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №3
по курсу «Программирование на языке
Java» на тему «Обработка исключительных
ситуаций»
Вариант 8

Выполнили:
студенты группы 23ВВП1
Батайкин
Д.В.
Володин
А.П.

Приняли:
Карамышева Н.С.
Юрова О.В.

Цель работы: изучить механизм обработки исключительных ситуаций.

Задание на лабораторную работу: Модифицировать приложение из предыдущей лабораторной работы, реализовав проверку вводимых данных с использованием механизма исключений. Необходимо создать свой класс, унаследованный от класса `Exception`, и генерировать исключение, если возникает попытка создать экземпляр класса `RecIntegral` со значениями, не являющимися числами в диапазоне от 0,000001 до 1000000. В качестве обработки исключения необходимо выводить диалог, содержащий предупреждение о некорректности введенных данных. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Ход работы:

1. Был создан отдельный класс `IntegralException` для обработки исключений, все ранее имеющиеся проверки и генерации исключений теперь выполняются с помощью него.
2. Была протестирована работа обработчика (Рис. 1-2).

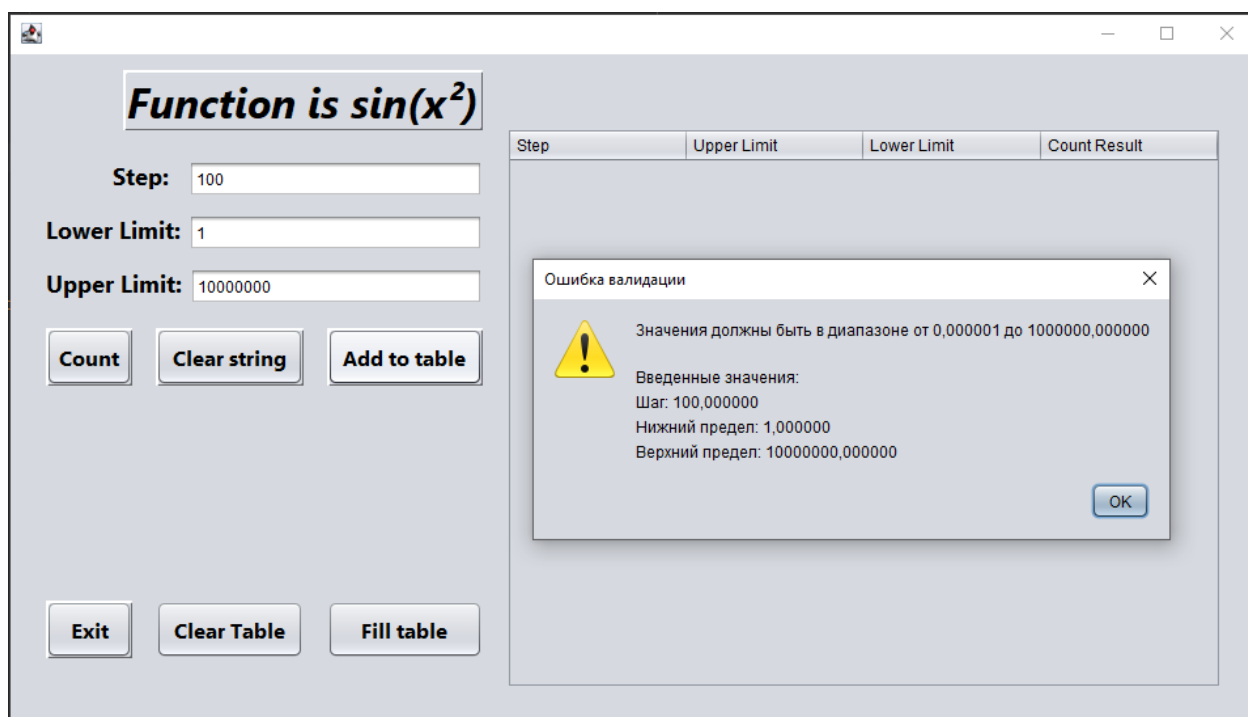


Рисунок 1 – Проверка обработки исключений в форме

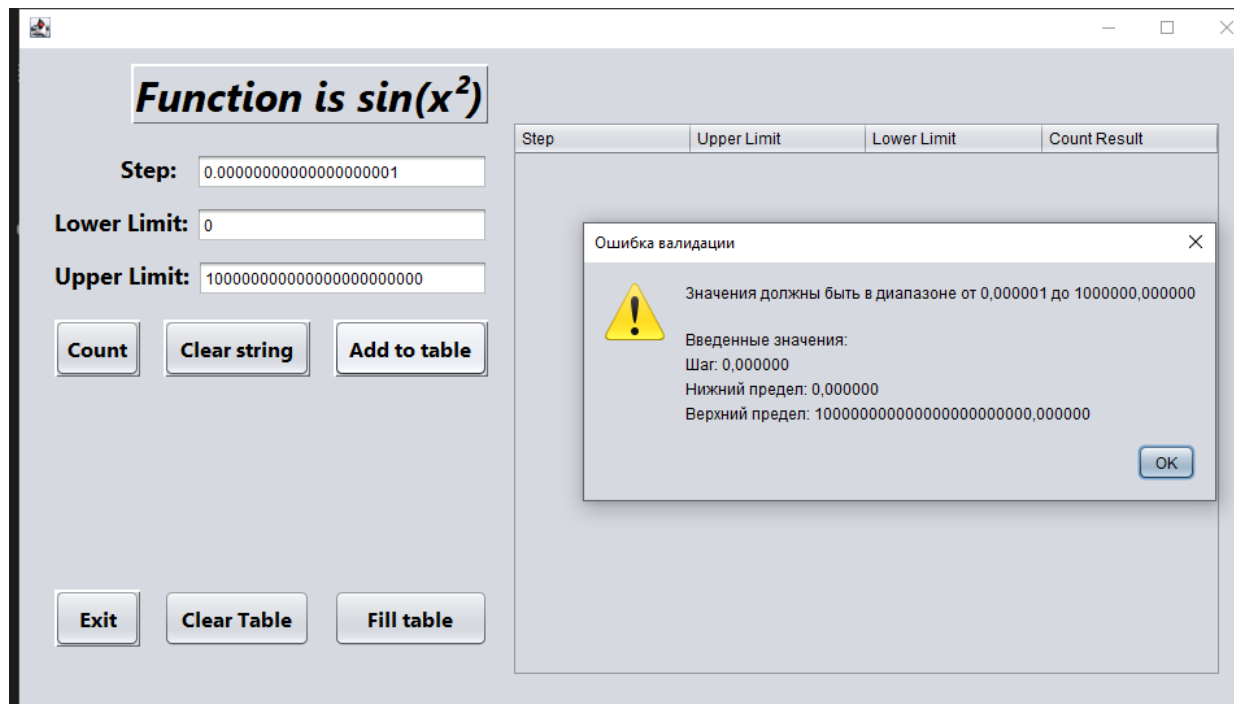


Рисунок 2 – Проверка обработки исключений в форме

Листинг программы:

Файл Lab3.java

```
package com.mycompany.lab3;

public class Lab3 {
    public static void main(String[] args) {
        java.awt.EventQueue.invokeLater(() -> {
            new Lab3JFrame().setVisible(true);
        });
    }
}
```

Файл Lab3JFrame.java

```
package com.mycompany.lab3;

import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import java.util.LinkedList;

public class Lab3JFrame extends javax.swing.JFrame {

    private static final java.util.logging.Logger logger =
        java.util.logging.Logger.getLogger(Lab3JFrame.class.getName());
    private LinkedList<RecIntegral> recordsList;

    public Lab3JFrame() {
        initComponents();
        setupTableFormatting();
        setupTableListener();
        recordsList = new LinkedList<>();
    }

    private void setupTableFormatting() {
        // Устанавливаем рендерер для отображения чисел
        javax.swing.table.DefaultTableCellRenderer rightRenderer = new
        javax.swing.table.DefaultTableCellRenderer();
        rightRenderer.setHorizontalAlignment(javax.swing.JLabel.RIGHT);

        // Применяем рендерер ко всем столбцам
        for (int i = 0; i < TableModel.getColumnCount(); i++) {
            TableModel.getColumnModel().getColumn(i).setCellRenderer(rightRenderer);
        }
    }

    private boolean isUpdating = false; // флаг для предотвращения рекурсивного вызова
    ошибки

    private void setupTableListener() {
        DefaultTableModel model = (DefaultTableModel) TableModel.getModel();

        model.addTableModelListener(e -> {
            if (isUpdating) return;

            if (e.getType() == javax.swing.event.TableModelEvent.UPDATE) {
                int row = e.getFirstRow();
                int column = e.getColumn();

                if (row >= 0 && column >= 0 && column <= 2 && row < recordsList.size()) {
                    isUpdating = true;
                    try {
                        RecIntegral record = recordsList.get(row);
                        // Сохраняем старые значения для восстановления
                        double oldStep = record.getStep();
                        double oldUpper = record.getUpperLimit();
                        double oldLower = record.getLowerLimit();
                    }
                }
            }
        });
    }
}
```

```

Object newValue = model.getValueAt(row, column);
if (newValue != null) {
    String valueStr = newValue.toString().trim().replace(',', ' ',
'.');

    if (!valueStr.isEmpty()) {
        double value = Double.parseDouble(valueStr);

        try {
            // Валидация и обновление с использованием класса
            switch (column) {
                case 0:
                    record.setStep(value);
                    break;
                case 1:
                    record.setUpperLimit(value);
                    break;
                case 2:
                    record.setLowerLimit(value);
                    break;
            }

            // проверка соотношения пределов
            if (record.getLowerLimit() >= record.getUpperLimit())
            {
                JOptionPane.showMessageDialog(this, "Нижний предел
должен быть меньше верхнего", "Ошибка", JOptionPane.WARNING_MESSAGE);

                // Восстанавливаем значения
                record.setStep(oldStep);
                record.setUpperLimit(oldUpper);
                record.setLowerLimit(oldLower);

                // Обновляем таблицу
                model.setValueAt(oldStep, row, 0);
                model.setValueAt(oldUpper, row, 1);
                model.setValueAt(oldLower, row, 2);
            } else {
                record.setResult(0.0);
                model.setValueAt("", row, 3);
            }
        } catch (IntegralException ex) {
            // Обработка исключения при редактировании ячейки
            String fieldName = "";
            double restoreValue = 0;

            switch (column) {
                case 0:
                    fieldName = "Шаг";
                    restoreValue = oldStep;
                    break;
                case 1:
                    fieldName = "Верхний предел";
                    restoreValue = oldUpper;
                    break;
                case 2:
                    fieldName = "Нижний предел";
                    restoreValue = oldLower;
                    break;
            }

            JOptionPane.showMessageDialog(this, String.format("Некорректное значение для поля '%s':
%.6f\n\n" + "Значение должно быть в диапазоне от %.6f до %.6f",
fieldName, value, RecIntegral.MIN_VALUE,
RecIntegral.MAX_VALUE), "Ошибка валидации", JOptionPane.WARNING_MESSAGE);
            // Восстанавливаем старое значение
            model.setValueAt(restoreValue, row, column);
        }
    }
}

```



```

        return canEdit [columnIndex];
    }
});
TableModel.setToolTipText("");
TableModel.setShowVerticalLines(true);
JScrollPane.setViewportViewView(TableModel);

CountButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); // NOI18N
CountButton.setText("Count");
CountButton.setBorder(new
javax.swing.border.SoftBevelBorder(javax.swing.border.BevelBorder.RAISED));
CountButton.addActionListener(this::CountButtonActionPerformed);

ClearStringButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); // NOI18N
ClearStringButton.setText("Clear string");
ClearStringButton.setBorder(new
javax.swing.border.SoftBevelBorder(javax.swing.border.BevelBorder.RAISED));
ClearStringButton.addActionListener(this::ClearStringButtonActionPerformed);

AddToTableButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); // NOI18N
AddToTableButton.setText("Add to table");
AddToTableButton.setBorder(new
javax.swing.border.SoftBevelBorder(javax.swing.border.BevelBorder.RAISED));
AddToTableButton.addActionListener(this::AddToTableButtonActionPerformed);

StepLabel.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
StepLabel.setText("Step:");

UpLimitLabel.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
UpLimitLabel.setText("Upper Limit:");

DownLimitLabel.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
DownLimitLabel.setText("Lower Limit:");

StepTextField.addActionListener(this::StepTextFieldActionPerformed);

UpperLimitTextField.addActionListener(this::UpperLimitTextFieldActionPerformed);

LowerLimitTextField.addActionListener(this::LowerLimitTextFieldActionPerformed);

ExitButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); // NOI18N
ExitButton.setText("Exit");
ExitButton.setBorder(new
javax.swing.border.SoftBevelBorder(javax.swing.border.BevelBorder.RAISED));
ExitButton.addActionListener(this::ExitButtonActionPerformed);

ClearTableButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); // NOI18N
ClearTableButton.setText("Clear Table");
ClearTableButton.addActionListener(this::ClearTableButtonActionPerformed);

FillTableButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); // NOI18N
FillTableButton.setText("Fill table");
FillTableButton.addActionListener(this::FillTableButtonActionPerformed);

javax.swing.GroupLayout jPanel2Layout = new javax.swing.GroupLayout(jPanel2);
jPanel2.setLayout(jPanel2Layout);
jPanel2Layout.setHorizontalGroup(

jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(jPanel2Layout.createSequentialGroup()
        .createSequentialGroup()
            .addGap(21, 21, 21)

    .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanel2Layout.createSequentialGroup()
            .addGap(21, 21, 21)

jPanel2Layout.createSequentialGroup()
    .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
        .addComponent(DownLimitLabel)

```

```

        .addComponent(StepLabel,
javax.swing.GroupLayout.PREFERRED_SIZE, 52, javax.swing.GroupLayout.PREFERRED_SIZE))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(LowerLimitTextField)
        .addComponent(StepTextField))
        .addGroup(jPanel2Layout.createSequentialGroup())
        .addComponent(UpLimitLabel)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(UpperLimitTextField))
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanel2Layout.createSequentialGroup())
        .addGap(0, 59, Short.MAX_VALUE)
        .addComponent(jLabel1))
        .addGroup(jPanel2Layout.createSequentialGroup())

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(CountButton,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
        .addComponent(ExitButton,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))
        .addGap(18, 18, 18)

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
        .addComponent(ClearStringButton,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
        .addComponent(ClearTableButton,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))
        .addGap(18, 18, 18)

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
        .addComponent(AddToTableButton,
javax.swing.GroupLayout.DEFAULT_SIZE, 119, Short.MAX_VALUE)
        .addComponent(FillTableButton,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)))
        .addGap(18, 18, 18)
        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE,
547, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(21, 21, 21))
);
jPanel2Layout.setVerticalGroup(

jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanel2Layout.createSequentialGroup())
        .addGap(50, 50, 50)
        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addContainerGap(20, Short.MAX_VALUE))
        .addGroup(jPanel2Layout.createSequentialGroup())
        .addContainerGap()
        .addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 46,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(23, 23, 23)

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
)
        .addComponent(StepTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(StepLabel, javax.swing.GroupLayout.PREFERRED_SIZE,
22, javax.swing.GroupLayout.PREFERRED_SIZE))

```



```

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

    .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    )
        .addComponent(DownLimitLabel)
        .addComponent(LowerLimitTextField,
        javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
        javax.swing.GroupLayout.PREFERRED_SIZE))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

    .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    )
        .addComponent(UpLimitLabel)
        .addComponent(UpperLimitTextField,
        javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
        javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(18, 18, 18)

    .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
    false)
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
        jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(AddToTableButton,
        javax.swing.GroupLayout.PREFERRED_SIZE, 44, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(ClearStringButton,
        javax.swing.GroupLayout.PREFERRED_SIZE, 44, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addComponent(CountButton,
        javax.swing.GroupLayout.Alignment.TRAILING, javax.swing.GroupLayout.PREFERRED_SIZE,
        44, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
        javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

    .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
    false)
        .addComponent(ExitButton, javax.swing.GroupLayout.PREFERRED_SIZE,
        44, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(FillTableButton,
        javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
        Short.MAX_VALUE)
        .addComponent(ClearTableButton,
        javax.swing.GroupLayout.Alignment.TRAILING, javax.swing.GroupLayout.DEFAULT_SIZE,
        javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
        .addGap(43, 43, 43)
    );

    javax.swing.GroupLayout jPanel1Layout = new javax.swing.GroupLayout(jPanel1);
    jPanel1.setLayout(jPanel1Layout);
    jPanel1Layout.setHorizontalGroup(

    jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanel1Layout.createSequentialGroup()
        .addContainerGap()
        .addComponent(jPanel2, javax.swing.GroupLayout.DEFAULT_SIZE,
        javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addContainerGap())
    );
    jPanel1Layout.setVerticalGroup(

    jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanel1Layout.createSequentialGroup()
        .addContainerGap()
        .addComponent(jPanel2, javax.swing.GroupLayout.DEFAULT_SIZE,
        javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addContainerGap())
    );

    javax.swing.GroupLayout layout = new
    javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

```

```

        .addComponent(jPanell1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(jPanell1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    pack();
} // </editor-fold>

private void CountButtonActionPerformed(java.awt.event.ActionEvent evt) {
    int selectedRow = TableModel.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(this, "Выберите строку для вычисления",
"Информация", JOptionPane.WARNING_MESSAGE);
        return;
    }
    try{
        DefaultTableModel model = (DefaultTableModel) TableModel.getModel();
        // Получаем значения из таблицы
        Double step = (Double) TableModel.getValueAt(selectedRow, 0);
        Double uplim = (Double) TableModel.getValueAt(selectedRow, 1);
        Double lowlim = (Double) TableModel.getValueAt(selectedRow, 2);

        // Проверяем на null
        if (step == null || uplim == null || lowlim == null) {
            JOptionPane.showMessageDialog(this, "Некоторые значения отсутствуют", "Ошибка",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        // Проверка корректности параметров
        if (lowlim >= uplim) {
            JOptionPane.showMessageDialog(this, "Нижний предел должен быть меньше
верхнего предела", "Ошибка параметров", JOptionPane.ERROR_MESSAGE);
            return;
        }
        if (step <= 0) {JOptionPane.showMessageDialog(this, "Шаг должен быть
положительным числом", "Ошибка параметров", JOptionPane.ERROR_MESSAGE);
            return;
        }

        double result = RecIntegral.computeIntegral(step, lowlim, uplim);

        // обновляем результат в коллекции
        if (selectedRow < recordsList.size()) {
            RecIntegral record = recordsList.get(selectedRow);
            record.setResult(result);
        }

        // Форматируем результат до 5 знаков
        model.setValueAt(String.format("%.5f", result), selectedRow, 3);
    }
    catch(ClassCastException e) {
        // обработка неверного формата
        JOptionPane.showMessageDialog(this, "Некорректный формат данных в
таблице", "Ошибка", JOptionPane.ERROR_MESSAGE);
    }
    catch(Exception e){
        JOptionPane.showMessageDialog(this, "Ошибка при вычислении: " +
e.getMessage(), "Ошибка", JOptionPane.ERROR_MESSAGE);
    }
}

private void ClearStringButtonActionPerformed(java.awt.event.ActionEvent evt) {
    int selectedRow = TableModel.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(this, "Выберите строку для удаления",
"Информация", JOptionPane.WARNING_MESSAGE);
        return;
    }

```

```

    }
    // получаем модель и удаляем строку из таблицы и коллекции
    DefaultTableModel model = (DefaultTableModel) TableModel.getModel();
    model.removeRow(selectedRow);
    recordsList.remove(selectedRow);
}

private void AddToTableButtonActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        String stepText = StepTextField.getText().trim().replace(',', '.'); //
        Поддержка запятой
        String lowlimText = LowerLimitTextField.getText().trim().replace(',', '.');
        String uplimText = UpperLimitTextField.getText().trim().replace(',', '.');

        // проверка на заполнение полей ввода
        if (stepText.isEmpty() || lowlimText.isEmpty() || uplimText.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Заполните все поля", "Ошибка",
            JOptionPane.ERROR_MESSAGE);
            return;
        }

        double step = Double.parseDouble(stepText);
        double lowlim = Double.parseDouble(lowlimText);
        double uplim = Double.parseDouble(uplimText);

        // Проверка диапазона значений (0.000001 до 1000000)
        if (step < RecIntegral.MIN_VALUE || step > RecIntegral.MAX_VALUE || lowlim
        < RecIntegral.MIN_VALUE || lowlim > RecIntegral.MAX_VALUE || uplim <
        RecIntegral.MIN_VALUE || uplim > RecIntegral.MAX_VALUE) {
            String message = String.format("Значения должны быть в диапазоне от
            %.6f до %.6f\n\n" + "Введенные значения:\n" + "Шаг: %.6f\n" + "Нижний предел: %.6f\n"
            + "Верхний предел: %.6f",
            RecIntegral.MIN_VALUE, RecIntegral.MAX_VALUE, step, lowlim, uplim);

            JOptionPane.showMessageDialog(this, message, "Ошибка валидации",
            JOptionPane.WARNING_MESSAGE);
            return;
        }

        // Создаем объект (конструктор без исключений)
        RecIntegral record = new RecIntegral(step, uplim, lowlim);

        // добавляем в список
        recordsList.add(record);
        // добавление в таблицу
        DefaultTableModel model = (DefaultTableModel) TableModel.getModel();
        model.addRow(record.toTableRow());

        // Очищаем поля
        StepTextField.setText("");
        LowerLimitTextField.setText("");
        UpperLimitTextField.setText("");
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
        "Введите корректные числовые значения (например: 0.1, 0.5, 1.0)",
        "Ошибка ввода",
        JOptionPane.ERROR_MESSAGE);
    }
}

private void StepTextFieldActionPerformed(java.awt.event.ActionEvent evt) {
}

private void UpperLimitTextFieldActionPerformed(java.awt.event.ActionEvent evt) {
}

private void LowerLimitTextFieldActionPerformed(java.awt.event.ActionEvent evt) {
}

```

```

    }

    private void ExitButtonActionPerformed(java.awt.event.ActionEvent evt) {
        System.exit(0);
    }

    private void ClearTableButtonActionPerformed(java.awt.event.ActionEvent evt) {
        DefaultTableModel model = (DefaultTableModel) TableModel.getModel();
        model.setRowCount(0);
    }

    private void FillTableButtonActionPerformed(java.awt.event.ActionEvent evt) {
        DefaultTableModel model = (DefaultTableModel) TableModel.getModel();

        if (recordsList.isEmpty()) {JOptionPane.showMessageDialog(this, "Коллекция
пуста. Сначала добавьте данные", "Информация", JOptionPane.INFORMATION_MESSAGE);
        return;}

        int rowCount = model.getRowCount();
        for (int i = rowCount - 1; i >= 0; i--) {
            model.removeRow(i);
        }
        for (RecIntegral record : recordsList) {
            model.addRow(record.toTableRow());
        }
    }

    public static void main(String args[]) {
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        }
        catch (Exception e) {
        }
        java.awt.EventQueue.invokeLater(() -> new Lab3JFrame().setVisible(true));
    }

    // Variables declaration - do not modify
    private javax.swing.JButton AddToTableButton;
    private javax.swing.JButton ClearStringButton;
    private javax.swing.JButton ClearTableButton;
    private javax.swing.JButton CountButton;
    private javax.swing.JLabel DownLimitLabel;
    private javax.swing.JButton ExitButton;
    private javax.swing.JButton FillTableButton;
    private javax.swing.JTextField LowerLimitTextField;
    private javax.swing.JLabel StepLabel;
    private javax.swing.JTextField StepTextField;
    private javax.swing.JTable TableModel;
    private javax.swing.JLabel UpLimitLabel;
    private javax.swing.JTextField UpperLimitTextField;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JPanel jPanel2;
    private javax.swing.JScrollPane jScrollPane1;
    // End of variables declaration
}

```

Файл RecIntegral.java

```
package com.mycompany.lab3;
```

```
public class RecIntegral {
    // константы минимального/максимального значений для вводимых данных
    public static final double MIN_VALUE = 0.000001;
    public static final double MAX_VALUE = 1000000.0;

    // Метод валидации значения
    private static void validateValue(double value, String fieldName) throws
IntegralException {
        if (value < MIN_VALUE || value > MAX_VALUE) {
            throw new IntegralException(String.format("Значение поля %s (%.6f) должно
быть в диапазоне от %.6f до %.6f", fieldName, value, MIN_VALUE, MAX_VALUE), fieldName,
value);
        }
    }

    // переменные таблицы
    private double step;
    private double upperLimit;
    private double lowerLimit;
    private double result;

    // конструктор по умолчанию
    public RecIntegral() {this.step = 0.0;
        this.upperLimit = 0.0;
        this.lowerLimit = 0.0;
        this.result = 0.0;
    }

    // Конструктор с параметрами
    public RecIntegral(double step, double upperLimit, double lowerLimit, double
result){
        this.step = step;
        this.upperLimit = upperLimit;
        this.lowerLimit = lowerLimit;
        this.result = result;
    }

    // Конструктор без результата (для новых записей)
    public RecIntegral(double step, double upperLimit, double lowerLimit) {
        this(step, upperLimit, lowerLimit, 0.0);
    }

    // геттеры и сеттеры переменных RecIntegral + проверка значений (валидация)
    public double getStep() { return step;}
    public void setStep(double step) throws IntegralException {
        validateValue(step, "Шаг");
        this.step = step;}

    public double getUpperLimit() {return upperLimit;}
    public void setUpperLimit (double upperLimit) throws IntegralException {
        validateValue(upperLimit, "Верхний предел");
        this.upperLimit = upperLimit;}
}
```

```

public double getLowerLimit() {return lowerLimit;}
public void setLowerLimit(double lowerLimit) throws IntegralException {
    validateValue(lowerLimit, "Нижний предел");
    this.lowerLimit = lowerLimit;}

public double getResult() {return result;}
public void setResult(double result) {this.result = result;}

    // Метод для преобразования в массив Object для таблицы
public Object[] toTableRow() {
    return new Object[]{step, upperLimit, lowerLimit, result == 0.0 ? "" :
String.format("%.5f", result)};
}

// Вычисление интеграла //
public static double func(double x) {
    return Math.sin(x*x);
}

    public static double computeIntegral(double step, double lowlim, double uplim) {
double sum = 0.0;
double x = lowlim;

while (x < uplim) {
    //Подсчет шага с учетом погрешности с помощью минимизации Math.min
    double currentstep = Math.min(step, uplim - x);
    double nextX = x + currentstep;

    double y1 = func(x);
    double y2 = func(nextX);
    // Площадь трапеции
    sum += (y1 + y2) * (nextX - x) / 2.0;
    x = nextX;
}
return sum;
}
}

```

Файл IntegralException.java

```

package com.mycompany.lab3;

public class IntegralException extends Exception {
    private String fieldName;    // поле, вызвавшее ошибку
    private double invalidValue; // значение, вызвавшее ошибку

    // конструктор
    public IntegralException(String message, String fieldName, double invalidValue) {
        // Передаем сообщение в родительский класс Exception
        super(message);
        this.fieldName = fieldName;
        this.invalidValue = invalidValue;
    }

    // конструктор когда необходим только текст об ошибке
    public IntegralException(String message) {super(message);}

    // геттеры
    public String getFieldName() {return fieldName;}

    public double getInvalidValue() {return invalidValue;}
}

```